

Année 2026 – 2027

TP 2 IOT



Etablissement scolaire : Polytech Sorbonne

Formation : Electronique et Informatique

Matières : IOT

Enseignant : Yan DOUZE

Table des matières

Remerciements.....	4
Résumé	5
I. Environnement de travail et outils utilisés	6
II. Objectif de ce tp.....	7
III. Ce que vous retrouverez dans les fichiers joints.....	7
IV. Configuration de The Things Network.....	8
V. Explication code esp32.....	9
VI. Explication node red	11
VII. Résultat.....	16
VIII. Annexe	16

Remerciements

Je tiens tout d'abord à remercier Monsieur Yan Douze et Monsieur Florian Dupeyron, qui nous ont encadrés tout au long de ce TP. Leur accompagnement, leurs conseils et leurs enseignements m'ont permis de mener à bien ce travail, tout en nous apportant le socle de connaissances nécessaire en IoT, notamment sur le LoRaWAN.

Résumé

Au sein de la formation d'ingénieur en Électronique et Informatique à Polytech Sorbonne, nous avons suivi l'enseignement Internet des Objets (IoT), qui nous a permis d'acquérir des connaissances fondamentales dans ce domaine. Cette matière nous a notamment introduits à la communication entre objets connectés, à l'utilisation de microcontrôleurs tels que l'ESP32, ainsi qu'aux différents protocoles de communication utilisés dans les systèmes IoT.

Cet enseignement m'a permis de mieux comprendre le fonctionnement global d'un système connecté, depuis l'acquisition des données par des capteurs jusqu'à leur transmission, leur traitement et leur visualisation. Il met également en évidence l'importance de la communication réseau, de la gestion des données et de l'interconnexion entre les différents équipements d'un système IoT.

I. Environnement de travail et outils utilisés

Dans le cadre de ce TP, j'ai choisi de travailler sur mon ordinateur personnel, car je suis plus à l'aise sur cet environnement et j'en maîtrise entièrement la configuration. Cela m'a permis de travailler plus efficacement, de mieux comprendre les éventuels problèmes de compilation ou de dépendances et de garder la maîtrise de l'ensemble de ma chaîne de développement.

J'ai utilisé Visual Studio Code avec l'extension PlatformIO (car je trouve que c'est mieux par rapport à Arduino IDE).

PlatformIO permet de développer et de téléverser des programmes sur des cartes comme l'ESP32 avec le framework Arduino. Le principe reste donc très proche de l'Arduino IDE : on écrit le programme, on le compile puis on le téléverse sur la carte. La différence principale est que PlatformIO s'intègre à VS Code et apporte une gestion de projet plus structurée, notamment pour les bibliothèques, les dépendances, les différentes cartes et les paramètres de compilation.

Je préfère également cette méthode car elle me laisse davantage de contrôle sur le projet. En particulier, il est possible d'ajouter des bibliothèques locales dans le dossier `lib/`, d'utiliser une version précise d'une dépendance ou, si nécessaire, de travailler sur une version modifiée d'une bibliothèque pour l'adapter au besoin du projet.

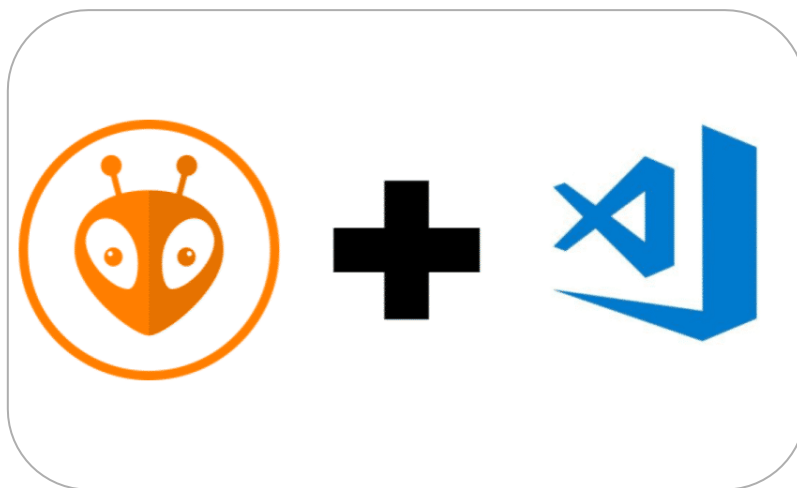


Figure 1 : Visualisation Platform IO + Visual Studio Code

Remarque :

Comme à mon habitude le projet se trouve sur mon compte github personnel à l'adresse suivante : https://github.com/Darkzvellier/TP_2_IOT_S7_polytech_sorbonne

II. Objectif de ce tp

L'objectif de ce TP est de prendre en main la communication LoRaWAN à l'aide d'un module LoRa-E5 connecté à un ESP32. Le but est dans un premier temps de comprendre comment fonctionne le module LoRa-E5, notamment à travers l'utilisation de commandes AT, puis de réussir à connecter le module au réseau The Things Network (TTN) et à envoyer des premières trames LoRaWAN.

Dans un second temps, l'objectif est d'automatiser l'envoi de données avec l'ESP32. Pour cela, un capteur DHT11 est utilisé afin de récupérer la température et l'humidité, puis ces valeurs sont envoyées vers TTN à travers le module LoRa-E5. L'idée est donc de ne plus envoyer les commandes manuellement depuis un PC, mais de laisser directement l'ESP32 communiquer avec le module via l'UART et construire les trames à transmettre.

Le TP permet ensuite de récupérer les données présentes sur TTN dans Node-RED grâce à MQTT, afin de les afficher et de pouvoir les exploiter plus facilement. Enfin, la dernière partie consiste à mettre en place une communication dans l'autre sens, c'est-à-dire un downlink, pour envoyer une commande depuis Node-RED vers l'ESP32 et par exemple allumer ou éteindre une LED à distance.

Au final, ce TP permet donc de comprendre toute la chaîne de communication, depuis la récupération d'une donnée sur l'ESP32 jusqu'à son envoi sur TTN et son affichage dans Node-RED, mais également de voir comment commander un équipement à distance grâce au réseau LoRaWAN.

III. Ce que vous retrouverez dans les fichiers joints

Dans les fichiers joints, vous retrouverez l'ensemble des éléments utilisés pour réaliser le TP, notamment le projet ESP32 LoRaWAN, la configuration de The Things Network, ainsi que les différents fichiers nécessaires pour reproduire le fonctionnement présenté dans ce compte rendu.

Le programme utilisé pour lancer directement certains outils n'est pas fourni, car il dépend de la configuration de mon ordinateur. En revanche, tous les fichiers importants permettant de refaire le TP sont présents.

Vous trouverez également un fichier *git_push.bat* que j'ai créé afin de simplifier l'envoi du projet sur Git. Celui-ci permet d'effectuer automatiquement les différentes étapes nécessaires,

notamment l'ajout des fichiers modifiés, la création du commit puis le push vers le dépôt distant, ce qui évite d'avoir à retaper manuellement toutes les commandes Git à chaque modification.

Voici l'arborescence générale du dossier :



Figure 2 : Arborescence générale du dossier

IV. Configuration de The Things Network

Avant de pouvoir envoyer des données, j'ai créé une application puis un **end device** dans The Things Network.

Dans la configuration du périphérique, le module a été associé au profil LoRaWAN de Seeed Technology correspondant au **LoRaWAN Dev Kit**, avec la région **EU_863_870** et le plan de fréquence européen **863-870 MHz**.

Le fonctionnement repose sur une activation **OTAA**. TTN fournit ou permet de définir les informations suivantes :

-  JoinEUI / AppEUI ;
-  DevEUI ;
-  AppKey ;

 identifiant du end device.

Ces valeurs doivent correspondre à celles envoyées au module LoRa-E5 par l'ESP32.

Dans le code, elles sont configurées avec des commandes de ce type :

```
envoyerCommande("AT+ID=AppEUI,0000000000000000");
envoyerCommande("AT+ID=DevEUI,70B3D57ED007922E");
envoyerCommande("AT+KEY=APPKEY,AF083CDC5BE612AAEB359989124B096B");
```

Figure 3 : code de configuration présent dans le code main.c

Le fichier Config_the_thing.docx contient les captures utilisées pendant cette étape. Le fichier The_things.url est simplement un raccourci vers la console du device et nécessite évidemment d'être connecté à un compte TTN ayant accès à l'application. Vous retrouverez en annexe les divers photos de chaque étape de la configuration.

V. Explication code esp32

Pour le programme de l'ESP32, j'ai essayé de séparer au maximum les différentes actions afin de rendre le code plus simple à comprendre. J'ai donc commencé par créer deux fonctions qui permettent de gérer la communication avec le module LoRa-E5. Ensuite, le setup() s'occupe de toute l'initialisation du système et enfin la loop() récupère les mesures du DHT11, les envoie sur TTN et vérifie si une commande a été reçue pour piloter la LED.

Avant cela, j'ai défini le DHT11 sur le GPIO 4 et la LED sur le GPIO 2. Le LoRa-E5 communique quant à lui avec l'ESP32 en UART grâce à Serial2.

La première fonction que j'ai créée est :

```
bool envoyerCommande(String commande, unsigned long timeout = 2000)...
```

Figure 4 : Prototype fonction evoyerCommande

Le but de cette fonction est simplement de pouvoir envoyer n'importe quelle **commande AT** au LoRa-E5 sans avoir à réécrire à chaque fois toute la partie permettant d'envoyer la commande et de récupérer sa réponse et d'avoir le retour de ces commandes.

Au début de la fonction, je vide d'abord ce qu'il peut rester dans le buffer de réception de Serial2.

Dans le code, il faut bien faire la différence entre les deux ports série. Serial correspond à la communication entre l'ESP32 et mon ordinateur, alors que Serial2 correspond à la communication entre l'ESP32 et le LoRa-E5. Je vérifie également, à l'aide d'un if, si le module m'a renvoyé "ERROR" ou "FAIL".

Pour la fonction lireReponseLoRa(), contrairement à la fonction précédente, celle-ci n'envoie aucune commande. Son rôle est simplement d'écouter ce que le LoRa-E5 envoie à l'ESP32. Je crée encore une fois une chaîne de caractères vide, puis je laisse une fenêtre de lecture de 5 secondes. Pendant ces 5 secondes, tous les caractères reçus sur Serial2 sont récupérés et également affichés sur le moniteur série. À la fin, la fonction retourne toute la réponse.

Cette fonction m'est surtout utile pour les downlinks. Après avoir envoyé une trame, je peux écouter le LoRa-E5 afin de savoir si TTN lui a renvoyé une commande, par exemple "01" ou "00" pour commander la LED.

Concernant le setup(), celui-ci est exécuté une seule fois au démarrage ou lors d'un redémarrage de l'ESP32. Je l'utilise donc pour initialiser tout ce qui est nécessaire avant de commencer les échanges.

Je commence par configurer la LED. Je fais ensuite un petit allumage au démarrage, surtout comme témoin visuel afin de voir directement que l'ESP32 a bien démarré et que le programme est en cours d'exécution.

J'initialise ensuite les deux ports série à une vitesse de 9600 bauds. Après cela, je teste la communication avec la commande AT afin de vérifier si le LoRa-E5 est bien présent sur l'UART. Cette commande m'a beaucoup aidé au début du TP pour vérifier rapidement si le câblage et la communication fonctionnaient correctement.

Ensuite, comme montré plus haut, je configure le LoRa-E5, j'active le mode OTAA, puis je le configure avec DR3, correspondant au SF9. Je demande ensuite au module de rejoindre le réseau avec la commande AT+JOIN. Une fois cela fait, j'envoie une petite trame de test afin de vérifier que la communication fonctionne bien, puis j'initialise le capteur DHT11.

Après cela, la loop() se lance.

Je commence par récupérer les valeurs d'humidité et de température du DHT11, puis je les affiche sur le moniteur série. Je multiplie ensuite les valeurs par 10, puis je crée une trame sous forme

hexadécimale en utilisant `sprintf()`. J'ai choisi ici de réserver 4 caractères hexadécimaux pour la température et 4 caractères pour l'humidité.

Une fois la trame créée, je l'envoie vers le LoRa-E5.

Juste après avoir envoyé le message, je récupère ce que le LoRa-E5 peut recevoir et j'affiche ensuite cette réponse dans le moniteur série.

C'est à ce moment-là que je vérifie si le downlink contient la valeur "01". La fonction `indexOf()` recherche simplement un morceau de texte à l'intérieur de toute la réponse reçue. Si "RX: "01"" est présent, `indexOf()` renvoie une position différente de -1, ce qui signifie que le message a bien été trouvé. Dans ce cas, j'allume la LED.

Je fais exactement le même principe avec "00", sauf que dans ce cas la LED est éteinte.

Enfin, j'ai ajouté un délai dans la `loop()`. Cependant, avec le délai de `lireReponseLoRa()` ainsi que ceux utilisés dans `envoyerCommande()`, cela fait au final beaucoup de temps d'attente et le fonctionnement devient assez bancal.

Il aurait été plus propre d'utiliser une méthode de type callback pour détecter directement lorsqu'une réponse est reçue, ou bien une fonction qui bufferise en continu les données reçues avant de faire le traitement. Cela permettrait d'éviter de bloquer inutilement le programme avec plusieurs délais.

VI. Explication node red

Pour Node-RED, j'ai réalisé un flow qui permet à la fois de récupérer les données envoyées par le LoRa-E5 depuis TTN, mais aussi de commander une LED à distance grâce à un downlink.

Pour la réception des données, j'utilise un nœud MQTT In directement connecté au broker MQTT de The Things Network. Celui-ci est abonné au topic correspondant à mon end device :

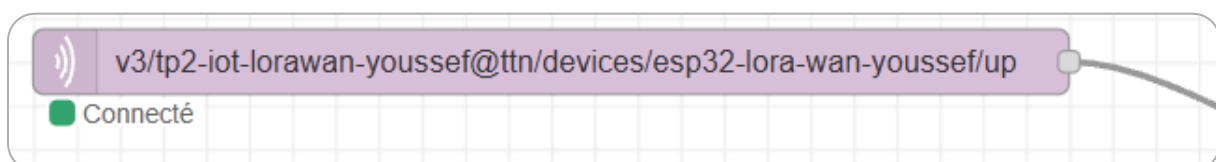


Figure 5 : Abonnement du topic

À chaque fois que l'ESP32 envoie une nouvelle trame LoRaWAN contenant la température et l'humidité, TTN reçoit la trame puis la transmet à Node-RED via MQTT. Le message reçu contient

plusieurs informations venant de TTN, dont les données qui ont déjà été décodées grâce au Payload Formatter.

J'utilise ensuite un premier nœud fonction pour ne récupérer que les informations qui m'intéressent, c'est-à-dire la température et l'humidité :

```

1  let data = msg.payload.uplink_message.decoded_payload;
2
3  msg.payload = {
4    temperature: data.temp,
5    humidite: data.hum
6  };
7
8  return msg;

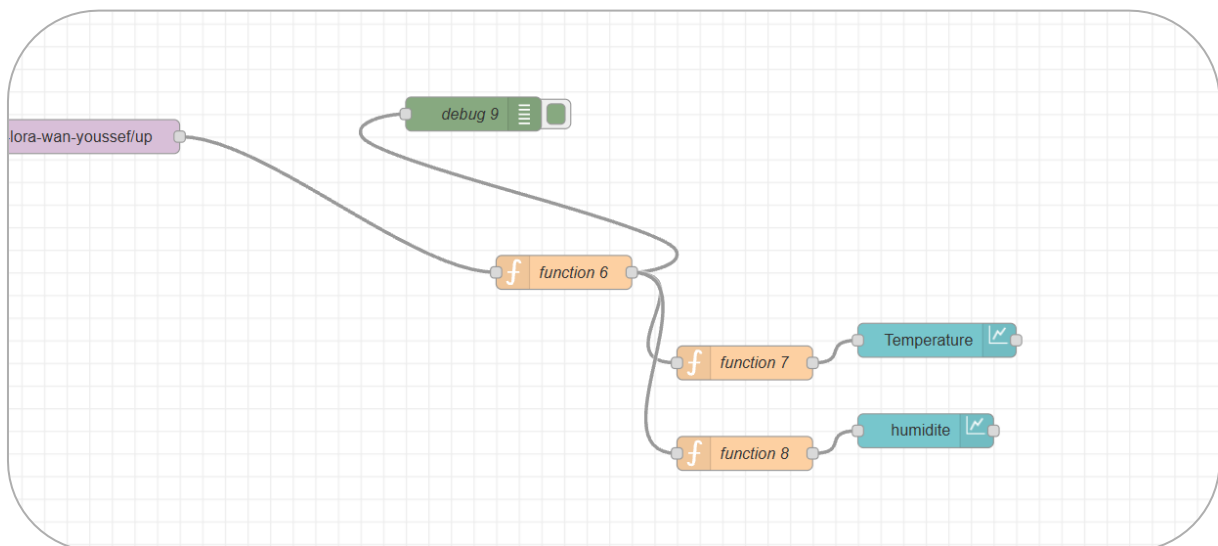
```

Figure 6 : Fonction node Red récupérant les information souhaités

À la sortie de cette fonction, j'obtiens donc un objet beaucoup plus simple.

Le flow est ensuite séparé en deux parties.

Une première fonction récupère uniquement la température et une autre l'humidité (respectivement fonction 7 et fonction 8).



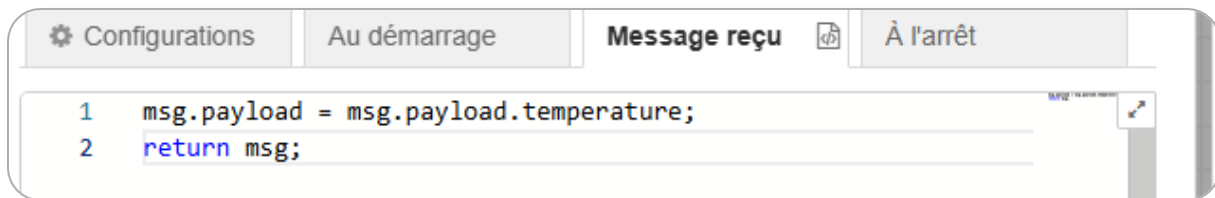


Figure 7 : Extrait du flow de l'extaction de données + extrait de code de la fonction 7

Ces valeur sont ensuite envoyées vers le graphique Temperatures et humidité du dashboard.

Cela me permet d'avoir sur le dashboard Node-RED deux graphiques qui évoluent automatiquement à chaque nouvelle mesure reçue depuis l'ESP32.

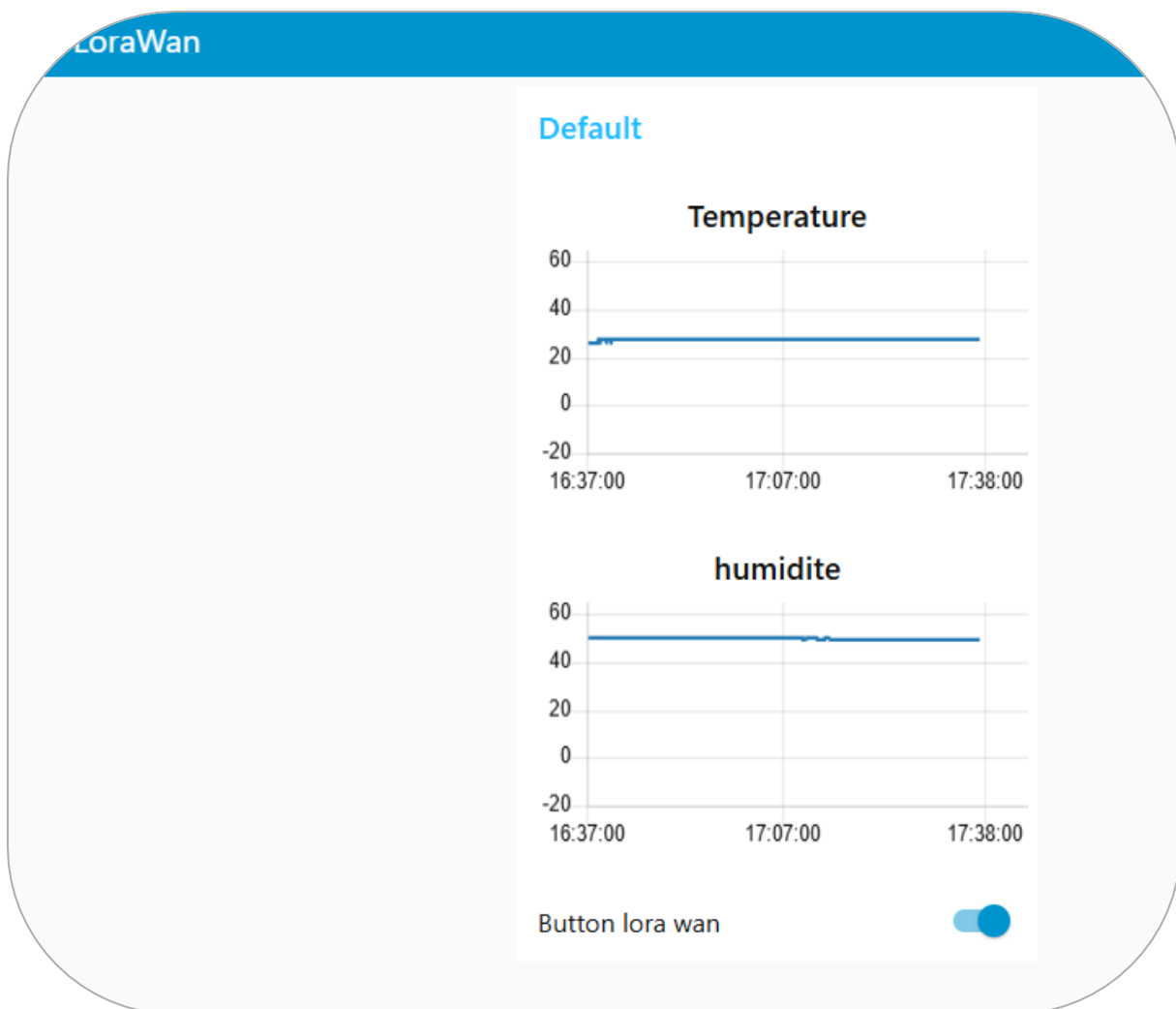


Figure 8 : Dashboard LoraWan

Pour la commande de la LED, j'ai ajouté un switch nommé Button lora wan sur le dashboard. Lorsque je change son état, il envoie une valeur correspondant à l'état que je souhaite donner à la LED, soit 1 ou 0.

Cette valeur est ensuite envoyée dans un autre nœud fonction qui prépare le message de downlink au format attendu par TTN.

```

1  let etat = Number(msg.payload);
2
3  let payload = Buffer.from([etat]).toString("base64");
4
5  msg.payload = JSON.stringify({
6    downlinks: [
7      {
8        f_port: 1,
9        frm_payload: payload,
10       priority: "NORMAL"
11      }
12    ]
13  });
14
15  return msg;

```

Figure 9 : Extrait code de la fonction 9 traduisant la valeur du bouton avant envoi vers TTN

Une fois le message préparé, il est envoyé vers un nœud MQTT Out, lui aussi connecté au broker de TTN, mais cette fois-ci sur le topic de downlink :

Modifier le noeud mqtt out

Supprimer Annuler Terminer

Propriétés

- Serveur: eu1.cloud.thethings.network:8883
- Sujet: v3/tp2-iot-lorawan-youssef@ttn/devices/esp32-lora-wan-youssef/down/push
- QoS: 0
- Conserver: [dropdown]
- Nom: Downlink_TTN_LED

Conseil : laisser le sujet, le qos ou le contenu vide si vous souhaitez les définir via les propriétés du msg.

Figure 10 : Ajout topic down-link

Le fonctionnement dans le sens de la remontée des données est donc le suivant :

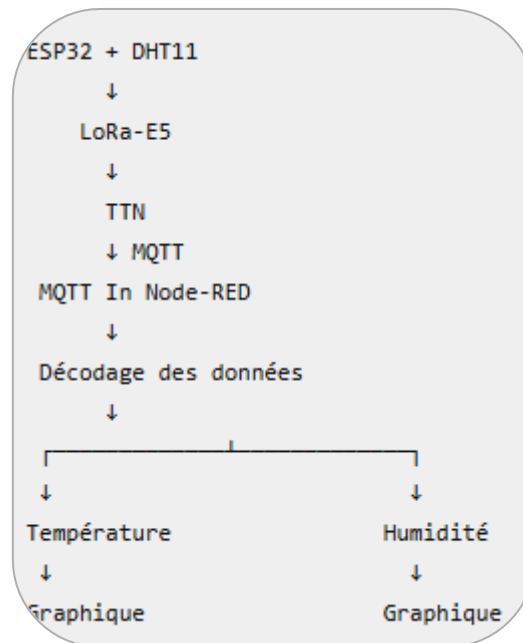


Figure 11 : Graphique expliquant la manière de communiquer

Et dans l'autre sens, pour la commande de la LED :

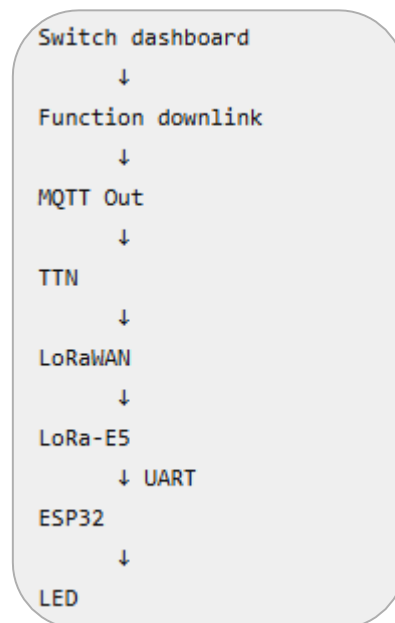


Figure 12 : Graphique expliquant la manière de communiquer en down link

Au final, cela me permet d'avoir un système bidirectionnel. L'ESP32 envoie les mesures du DHT11 vers TTN puis Node-RED afin de pouvoir les visualiser, et dans l'autre sens, Node-RED peut envoyer une commande jusqu'à l'ESP32 afin de modifier l'état de la LED.

VII. Résultat

Pour la partie résultat, la démonstration a été validé .

VIII. Annexe

Register end device

Does your end device have a LoRaWAN® Device Identification QR Code? Scan it to speed up onboarding.

 Scan end device QR code

 [Device registration help](#)

End device type

Input method

- ☒ Select the end device in the LoRaWAN Device Repository
- ☐ Enter end device specifics manually

End device brand  *

Seeed Technolog... | ▾

Model  *

LoRaWAN Dev Kit | ▾

Hardware Ver.  *

1.0 | ▾

Firmware Ver.  *

1.0 | ▾

Profile (Region) *

EU_863_870 | ▾

LoRaWAN Dev Kit

LoRaWAN Specification 1.0.3, RP001 Regional Parameters 1.0.3 revision A, Over the air activation (OTAA), Class A



The kit includes a Wio Terminal master controller with WIFI/BLE and display, as well as carefully selected Grove - LoRa-E5, Grove - VOC and eCO2 Gas Sensor SGP30, Grove - Soil Moisture Sensor, Grove - Temp&Humi Sensor (SHT40), and Grove - Vision AI Module. Humi Sensor (SHT40), and Grove - Vision AI Module. This kit is a perfect combination of LoRa and Edge AI, which allows software engineers to quickly deploy their first LoRaWAN projects through plug-and-play connectivity.

[Product website](#) | [Data sheet](#)

Frequency plan *

Europe 863-870 MHz (SF9 for RX2 - recommended) | ▾

Informations de provisionnement

JoinEUI ⓘ *

00 00 00 00 00 00 00 00

Réinitialiser

Ce terminal peut être enregistré sur le réseau

DevEUI ⓘ *

70 B3 D5 7E D0 07 92 2E

Générer

1/50 utilisé

Clé d'application ⓘ *

AF 08 3C DC 5B E6 12 AA EB 35 99 89 12 4B 09 6B

Générer

ID du périphérique final ⓘ *

esp32-lora-wan-youssef

Propriétés de l'appareil

Étiquettes

Sélectionner...



Ajoutez une étiquette ou [créez-en une](#) pour catégoriser vos appareils

Après l'inscription

- ☒ Afficher le dispositif final enregistré
- ☐ Enregistrez un autre terminal de ce type.

Enregistre le périphérique final